

数据结构

并查集
堆
单调队列
ST表
树状数组
线段树
扫描线

数据结构

并查集

定义： 每棵树为一个集合

- **合并操作：** 合并两个元素所属集合
- **查询：** 查询某个元素所属集合, 判断两元素是否属于同一个集合

路径压缩： 破坏了树的结构, 将经过的元素直接连在根节点以加快查询, 最坏时间复杂度 $O(m \log n)$

```
1 struct DSU : vector<signed> {
2     DSU(int n) : vector(n) { iota(begin(), end(), 0); }
3     int find(int u) {
4         for ( ; at(u) != u; u = at(u) = at(at(u)));
5         return u;
6     }
7     int merge(int u, int v) { return find(u) == find(v) ? -1 : at(find(u)) = find(v); }
8 };
```

启发式合并： 不破坏原有结构, 将秩小的并查集合并到秩大的并查集, 最坏时间复杂度 $O(m \log n)$

```
1 vector<int> fa(n + 1), r(n + 1, 1);
2 iota(fa.begin(), fa.end(), 0); //初始化另一种写法
3 function<int(int)> find = [&](int u) -> int {
4     return fa[u] == u ? u : find(fa[u]);
5 };
6 for (int i = 1, u, v; i <= m; i++) {
7     cin >> u >> v;
8     u = find(u), v = find(v);
9     if (u == v) {
10         continue;
11     } else if (r[u] > r[v]) {
12         swap(u, v);
13     }
14     fa[u] = v, r[v] += r[u];
15 }
```

带权并查集：

```
1 vector<int> dep(n + 1); // 维护深度, 要保证根节点深度为 0, 防止查询时重复计算贡献
2 auto find = [&](auto && find, int u) -> int {
3     if (fa[u] == u) {
4         return u;
```

```

5     }
6     int t = find(find, fa[u]);
7     dep[u] += dep[fa[u]];
8     return fa[u] = t;
9 };
10
11 auto merge = [&](int u, int v) {
12     int fu = find(find, u), fv = find(find, v);
13     if (fu != fv) {
14         fa[fv] = u;
15         dep[v] = 1;
16         find(find, v);
17     }
18 };

```

种类并查集：

堆

二叉堆： 树根存最大值为大根堆, 树根存最小值为小根堆

对顶堆： 由一个大根堆和小根堆组成, 大根堆维护比第 k 个大数小的数, 小根堆维护当前 k 大的值

使用堆时可能需要重载运算符

时间复杂度 $O(\log n)$

```

1     struct node {
2         int x, y;
3         bool operator<(const node& N) const {
4             return x < N.x;
5         }
6         bool operator>(const node& N) const {
7             return x > N.x;
8         }
9     };
10
11 auto cmp = [&](const node a, const node b) {
12
13 };
14
15 priority_queue<node, vector<node>, decltype(cmp)> pque3(cmp);
16
17 priority_queue<int, vector<int>, less<int>> pque1; //大根堆
18 priority_queue<int, vector<int>, greater<int>> pque2; //小根堆
19 if (pque1.size() == 0 || pque1.top() >= x) { //对顶堆
20     pque1.push(x);
21 } else {
22     pque2.push(x);
23 }

```

单调队列

```
1 struct binqueue : deque<array<int, 2>> {
2     void push(const array<int, 2> & t) {
3         for ( ; size() && t[1] < back()[1]; pop_back());
4         emplace_back(t);
5     } // chk 保证队列的范围
6     template <class F> void pop(F && chk) { for ( ; size() && !chk(front()[0]);
7         pop_front()); }
8     };
```

ST表

定义：ST表是用于解决可重复贡献问题的数据结构

```
1 int m = ceil(log2(n)) + 1; //加一是为了开大空间防re
2 vector<int> a(n);
3 vector<vector<int>> st(n, vector<int>(m));
4 for (int i = 0; i < n; i++) {
5     st[i][0] = a[i];
6 }
7 for (int j = 1; j < m; j++) { //init
8     int d = 1 << j - 1; //判断是否越界
9     for (int i = 0; i + d < n; i++) {
10         st[i][j] = op(st[i][j - 1], st[i + (1 << j - 1)][j - 1]);
11     }
12 }
13
14 int query(int l, int r) {
15     if (l == r) {
16         return a[l];
17     }
18     int t = r - l + 1;
19     t = ceil(log2(t)) - 1; //减一是提前减一
20     return op(st[l][t], st[r - (1 << t) + 1][t]);
21 }
```

树状数组

作用：支持单点修改和区间查询

将 $[1, n]$ 拆分为不多于 $\log n$ 段区间

空间复杂度 $O(n)$, 单次查询或修改时间复杂度 $O(\log n)$

```
1 auto lowbit = [](int x) {
2     return x & -x;
3 };
4
5 vector<int> tr(n + 1);
6
7 auto add = [&](int x, int k) {
8     for ( ; x < tr.size(); x += lowbit(x)) {
9         tr[x] += k;
10     }
```

```

10     }
11 };
12
13 auto get = [&](int x, int res = 0) {
14     for ( ; x; x -= lowbit(x)) {
15         res += tr[x];
16     }
17     return res;
18 };

```

区间修改、单点查询： 维护差分数组

区间修改、区间查询：

维护差分数组 d_i 和数组 $i \times d_i \Rightarrow \sum_{i=l}^r a_i = ((r+1) \times \sum_{i=1}^r d_i - l \times \sum_{i=1}^{l-1} d_i) - \sum_{i=l}^r (i \times d_i)$

固定一端的区间和的异或和： 转化为形如 $\bigoplus_{i=0}^n (p_n - p_i)$ ，对于第 bit 位，等价于查询

$(p_n - p_i) \bmod 2^{bit+1} \geq 2^{bit}$ 的个数，然后分类讨论，分成 $p_n - p_i < 0, 0 \leq p_n - p_i < 2^{bit+1}$ 和 $p_n - p_i \geq 2^{bit+1}$ 三种情况查询即可

线段树

作用： 可在 $O(\log N)$ 内实现单点修改，区间查询，区间修改(区间求和，求区间最大值，最小值)等

将每个长度不为1的区间划分成左右两个区间递归求解，通过合并左右区间来得答案，也可进行区间操作

最后的叶子节点即为原数组的信息，由于采用堆式存储，叶子节点为 n 时，数组范围最大为

$2^{\lceil \log_2 n \rceil + 1} \leq 4n - 5$

(若使用动态开点，则数组范围最大为 $2n - 1$)

故空间复杂度为 $O(n)$

- **区间加某个值与区间修改为某个值：**

```

1  template <class Info, class Tag>
2  struct SegTree : vector<Info> {    // 左闭右开，注意初始化数组需要有 1, r
3      using tr = vector<Info>;
4      int n;
5      vector<Tag> tag;
6
7      void init(const tr & t) { // t.size() > 0
8          n = t.size(), tr::assign(4 << __lg(t.size()), {}), tag.assign(tr::size(), {});
9          auto self = [&](auto && self, int l, int r, int p = 0) {
10              if (l + 1 == r) return tr::at(p) = t[l], void();
11              int m = l + r >> 1;
12              self(self, l, m, p << 1 | 1), self(self, m, r, p + 1 << 1), pull(p);
13          };
14          self(self, 0, t.size());
15      }
16      SegTree(int n) {
17          tr t(max<int>(1, n));
18          for (int i = 0; i < t.size(); i++) t[i] = Info(i, i + 1);
19          init(t);
20      }
21      SegTree(const tr & t) { init(t); }
22
23      void pull(int p) { tr::at(p).pull(tr::at(p << 1 | 1) + tr::at(p + 1 << 1)); }

```

```

24     void update(int p, const Tag & t) {
25         if (t != Tag()) tr::at(p).update(t), tag[p] += t;
26     }
27     void push(int p) {
28         if (tag[p] != Tag()) update(p << 1 | 1, tag[p]), update(p + 1 << 1, tag[p]),
tag[p] = {};
29     }
30
31     void update(int b, int e, const Tag & t) {
32         auto self = [&](auto && self, int p, int l, int r) -> void {
33             if (e <= l || r <= b) return;
34             if (b <= l && r <= e) return update(p, t);
35             int m = l + r >> 1;
36             push(p), self(self, p << 1 | 1, l, m), self(self, p + 1 << 1, m, r),
pull(p);
37         };
38         return self(self, 0, 0, n);
39     }
40     Info query(int b, int e) {
41         auto self = [&](auto && self, int p, int l, int r) -> Info {
42             if (e <= l || r <= b) return Info(b, e);
43             if (b <= l && r <= e) return tr::at(p);
44             int m = l + r >> 1;
45             return push(p), self(self, p << 1 | 1, l, m) + self(self, p + 1 << 1, m,
r);
46         };
47         return self(self, 0, 0, n);
48     }
49     template <class F>
50     int FindFirst(int b, int e, F && chk) {
51         auto self = [&](auto && self, int p, int l, int r) -> int {
52             if (e <= l || r <= b || !chk(tr::at(p))) return -1;
53             if (l + 1 == r) return l;
54             int m = l + r >> 1, res = -1;
55             return push(p), (res = self(self, p << 1 | 1, l, m)) == -1 ? self(self, p
+ 1 << 1, m, r) : res;
56         };
57         return self(self, 0, 0, n);
58     }
59     template <class F>
60     int FindLast(int b, int e, F && chk) {
61         auto self = [&](auto && self, int p, int l, int r) -> int {
62             if (e <= l || r <= b || !chk(tr::at(p))) return -1;
63             if (l + 1 == r) return l;
64             int m = l + r >> 1, res = -1;
65             return push(p), (res = self(self, p + 1 << 1, m, r)) == -1 ? self(self, p
<< 1 | 1, l, m) : res;
66         };
67         return self(self, 0, 0, n);
68     }
69 };
70
71 template <class Info, class Tag>
72 struct DSegTree : vector<Info> { // 多一个 chknod 可以开值域大小
73     using tr = vector<Info>;
74     int l, r;
75     vector<array<signed, 2>> son;
76     DSegTree(int l, int r) : l(l), r(r) { node(l, r); }
77     int node(int l, int r, int p = -1) {

```

```

78         if (p != -1) return p;
79         return tr::emplace_back(l, r), son.emplace_back(array{-1, -1}), son.size() -
1;
80     }
81     void pull(int p) { tr::at(p).pull(tr::at(son[p][0]) + tr::at(son[p][1])); }
82
83     void update(int b, int e, const Tag & t, int p = 0) {
84         auto self = [&](auto && self, int p, int l, int r) -> int {
85             if ((p = node(l, r, p)), e <= l || r <= b) return p;
86             if (b <= l && r <= e) return tr::at(p).update(t), p;
87             int m = l + r >> 1;
88             son[p][0] = self(self, son[p][0], l, m), son[p][1] = self(self, son[p][1],
m, r);
89             return pull(p), p;
90         };
91         self(self, p, l, r);
92     }
93     Info query(int b, int e, int p) {
94         auto self = [&](auto && self, int p, int l, int r) -> Info {
95             if (p == -1 || e <= l || r <= b) return Info(l, r);
96             if (b <= l && r <= e) return tr::at(p);
97             int m = l + r >> 1;
98             return self(self, son[p][0], l, m) + self(self, son[p][1], m, r);
99         };
100         return self(self, p, l, r);
101     }
102     template <class F>
103     int FindFirst(int b, int e, F && chk, int p) {
104         auto self = [&](auto && self, int p, int l, int r) -> int {
105             if (p == -1 || e <= l || r <= b || !chk(tr::at(p))) return -1;
106             if (l + 1 == r) return l;
107             int m = l + r >> 1, res = -1;
108             return (res = self(self, son[p][0], l, m)) == -1 ? self(self, son[p][1],
m, r) : res;
109         };
110         return self(self, p, l, r);
111     }
112     template <class F>
113     int FindLast(int b, int e, F && chk, int p) {
114         auto self = [&](auto && self, int p, int l, int r) -> int {
115             if (p == -1 || e <= l || r <= b || !chk(tr::at(p))) return -1;
116             if (l + 1 == r) return l;
117             int m = l + r >> 1, res = -1;
118             return (res = self(self, son[p][1], m, r)) == -1 ? self(self, son[p][0],
l, m) : res;
119         };
120         return self(self, p, l, r);
121     }
122     pair<int, int> merge(int t, int p) { // tie(t, p) = merge(t, p), t 会清空为 -1
123         auto self = [&](auto && self, int t, int p, int l, int r) -> int {
124             if (t == -1 || p == -1) return node(l, r, max(t, p));
125             if (l + 1 == r) return tr::at(p) += tr::at(t), p;
126             int m = l + r >> 1;
127             son[p][0] = self(self, son[t][0], son[p][0], l, m), son[p][1] = self(self,
son[t][1], son[p][1], m, r);
128             return pull(p), p;
129         };
130         return {-1, self(self, t, p, l, r)};
131     }

```

```

132     pair<int, int> split(int b, int e, int p) { // tie(t, p) = split(b, e, p);
133         auto self = [&](auto && self, int p, int l, int r, int t = -1) -> pair<int,
134         int> {
135             if ((t = node(l, r, t)), p == -1 || e <= l || r <= b) return {t, node(l,
136             r, p)};
137             if (b <= l && r <= e) return {p, t};
138             int m = l + r >> 1;
139             tie(son[t][0], son[p][0]) = self(self, son[p][0], l, m), tie(son[t][1],
140             son[p][1]) = self(self, son[p][1], m, r);
141             return pull(p), pull(t), pair{t, p};
142         };
143         return self(self, p, l, r);
144     };
145 };
146
147 namespace mul_add {
148     int mod;
149     struct Info {
150         unsigned l, r;
151         int sum;
152         Info(int l = -1, int r = -1, int sum = 0) : l(l), r(r), sum(sum) {}
153         struct Tag {
154             int mul = 1, add = 0;
155             Tag & operator += (const Tag & t) {
156                 return (mul *= t.mul) %= mod, (add = add * t.mul + t.add) %= mod,
157                 *this;
158             } Tag operator + (const Tag & t) const { return Tag(*this) += t; }
159             bool & operator == (const Tag & t) const {
160                 return mul == t.mul && add == t.add;
161             } bool & operator != (const Tag & t) const { return !(*this == t); }
162         };
163         Info & operator += (const Info & t) {
164             return r = t.r, (sum += t.sum) %= mod, *this;
165         } Info operator + (const Info & t) const { return Info(*this) += t; }
166         void pull(const Info & t) { *this = t; } // 是拉取的信息，一般等于即可
167         void update(const Tag & t) { (sum = sum * t.mul + (r - l) * t.add) %= mod; }
168     };
169 }
170
171 namespace Scan_Line {
172     struct Info {
173         int l, r;
174         int cnt, len, sum; // cnt 为永久化标记
175         Info(int l = -1, int r = -1, int cnt = 0, int len = 0, int sum = 0) : l(l),
176         r(r), cnt(cnt), len(len), sum(sum) {}
177         struct Tag {
178             int add = 0; // 扫描线标记不下传
179             Tag & operator += (const Tag & t) {
180                 return *this;
181             } Tag operator + (const Tag & t) const { return Tag(*this) += t; }
182             bool & operator == (const Tag & t) const {
183                 return add == t.add;
184             } bool & operator != (const Tag & t) const { return !(*this == t); }
185         };
186         Info & operator += (const Info & t) {
187             return r = t.r, sum = (len += t.len), *this;
188         } Info operator + (const Info & t) const { return Info(*this) += t; }
189         void pull(const Info & t) { *this = {t.l, t.r, cnt, cnt ? t.r - t.l : t.sum,
190         t.sum}; }

```

```

185         void update(const Tag & t) { cnt += t.add, len = cnt ? r - 1 : sum; }
186     };
187 }

```

权值线段树:

一般需要离散化, 维护左闭右闭的区间, 一般相当于一个桶

查询最大的小于 x 的数可以先用 *query* 查询 x 的排名 $rank$, 然后查询全局第 $rank - cnt(x)$ 大的数即可

查询最小的大于 x 的数同理

主席树(可持久化线段树):

- 查询区间答案 $ans[l, r] = \begin{cases} ans[nxt_l + 1, r], & r \geq nxt_l \\ ans[l + 1, r], & otherwise \end{cases}$, 令 $ans[l]$ 是左端点为 l 的答案序列, $ans[l]$ 可以由一部分的 $ans[l + 1]$ 和另一部分的 $ans[nxt_l + 1]$ 拼接而成, 维护 $ans[l + 1]$ 和 $ans[nxt_l + 1]$ 的历史版本合并成新的版本

```

1  struct node {
2      int lson, rson, cnt;    //记录左儿子,右儿子,出现次数
3  };
4
5  vector<node> tr((n << 2) + (n * 18));    //空间为n * 4 + n log n
6
7  vector<int> root(n + 1);    //每个历史版本的根
8  int tot = 0;
9
10 vector<int> a(n + 1);
11
12 for (int i = 1; i <= n; i++) {
13     cin >> a[i];
14 }
15
16 auto ta = a;
17
18 sort(ta.begin() + 1, ta.end());
19
20 ta.erase(unique(ta.begin() + 1, ta.end()), ta.end());    //离散化
21
22 auto get_val = [&](int x) {
23     return lower_bound(ta.begin() + 1, ta.end(), x) - ta.begin();
24 };
25
26 auto pushup = [&](int p) {
27     tr[p].cnt = tr[tr[p].lson].cnt + tr[tr[p].rson].cnt;
28 };
29
30 auto build = [&](auto build, int l, int r) -> int {
31     int root = ++tot;    //新开节点
32     if (l == r) return root;
33     int mid = l + r >> 1;
34     tr[root] = {build(build, l, mid), build(build, mid + 1, r)};
35     return root;    //返回节点编号
36 };
37
38 auto insert = [&](auto insert, int l, int r, int k, int root) -> int {    //k为插入点的权
    值,dir为新节点
39     int dir = ++tot;
40     tr[dir] = tr[root];    //新开节点并继承
41     if (l == r) {

```

```

42         tr[dir].cnt++;
43         return dir;
44     }
45     int mid = l + r >> 1;
46     if (k <= mid) tr[dir].lson = insert(insert, l, mid, k, tr[root].lson); //在左边向左边
插
47     else tr[dir].rson = insert(insert, mid + 1, r, k, tr[root].rson); //在右边向右边插
48     pushup(dir); //向上传
49     return dir; //返回节点编号
50 };
51
52 auto query = [&](auto query, int root, int dir, int l, int r, int k) -> int { //root为
l-1的节点,dir为r的节点
53     if (l == r) return r;
54     int cnt = tr[tr[dir].lson].cnt - tr[tr[root].lson].cnt; //前缀和思想
55     int mid = l + r >> 1;
56     if (k <= cnt) return query(query, tr[root].lson, tr[dir].lson, l, mid, k); //在左边
57     else return query(query, tr[root].rson, tr[dir].rson, mid + 1, r, k - cnt); //在右边,
且k-cnt
58 };
59
60 root[0] = build(build, 1, ta.size() - 1); //第一个历史版本
61
62 for (int i = 1; i <= n; i++) {
63     root[i] = insert(insert, 1, ta.size() - 1, get_val(a[i]), root[i - 1]);
64 }
65
66 for (int i = 1; i <= m; i++) {
67     int l, r, k;
68     cin >> l >> r >> k;
69     cout << ta[query(query, root[l - 1], root[r], 1, ta.size() - 1, k)] << "\n";
70 }

```

扫描线

作用：扫描面积、周长和二维数点

扫描线不需要 *pushdown* 和 *query*，维护的为一段左闭右开的区间

Atlantis:

```

1  vector<array<int, 4>> seg(1);
2  vector<int> X(1);
3
4  for (int i = 1; i <= n; i++) {
5      int x1, y1, x2, y2;
6      cin >> x1 >> y1 >> x2 >> y2;
7      seg.push_back({x1, x2, y1, 1}); //加边
8      seg.push_back({x1, x2, y2, -1}); //减边
9      X.push_back(x1), X.push_back(x2);
10 }
11
12 sort(seg.begin() + 1, seg.end(), [&](auto x, auto y) { //离散化
13     return x[2] < y[2]; //当纵坐标相同时先计算加边再计算减边
14 });
15 sort(X.begin() + 1, X.end());
16 X.erase(unique(X.begin() + 1, X.end()), X.end());
17

```

```

18 struct node {
19     int l, r;
20     array<int, 2> LR = {0}; //扫描线维护的为线段
21     int sum = 0, add = 0; //需要赋初值, 赋为幺元
22 };
23
24 vector<node> tr;
25
26 auto merge = [&](node l, node r, node u = {}) { //合并贡献
27     u = {l.l, r.r, u.LR, l.sum + r.sum, u.add}; //记得补全
28     return u;
29 };
30
31 auto pushup = [&](int p) {
32     if (tr[p].add) { //线段已经被覆盖
33         tr[p].sum = tr[p].LR[1] - tr[p].LR[0];
34     } else if (tr[p].r == tr[p].l) { //叶子节点, 不从子节点得到答案, 防止越界
35         tr[p].sum = 0;
36     } else { //非叶子节点, 从子节点获取答案
37         tr[p] = merge(tr[p << 1], tr[p << 1 | 1], tr[p]); //向上传
38     }
39 };
40
41 auto add_down = [&](int k, int p) { //加法的更新
42     tr[p].add += k;
43     pushup(p); //更新, 扫描线的更新与pushup相同
44 };
45
46 auto build = [&](int sz) { //建树
47     tr = vector<node>(sz << 2);
48
49     auto build = [&](auto build, int l, int r, int p = 1) {
50         tr[p] = {l, r, {X[l], X[r + 1]}};
51         if (l == r) { //叶子节点等于原数组
52             return;
53         }
54         int mid = l + r >> 1;
55         build(build, l, mid, p << 1), build(build, mid + 1, r, p << 1 | 1);
56         pushup(p);
57     };
58
59     build(build, 1, sz);
60 };
61
62 auto update = [&](auto update, int l, int r, int k, int p = 1) { //更新
63     if (l <= tr[p].LR[0] && tr[p].LR[1] <= r) {
64         add_down(k, p); //添加标记, 更新节点
65         return;
66     }
67     if (l < tr[p << 1].LR[1]) update(update, l, r, k, p << 1); //左边有交集, 因为为线段, 相
68     //等时无交集
69     if (tr[p << 1 | 1].LR[0] < r) update(update, l, r, k, p << 1 | 1); //右边有交集
70     pushup(p);
71 };
72
73 build(X.size() - 2); //线段树节点为区间, 坐标最后一个点不能用
74
75 int ans = 0, y0 = seg[1][2];

```

```

76 for (int i = 1; i < seg.size(); i++) {
77     auto [x1, x2, y, op] = seg[i];
78     ans += tr[1].sum * (y - y0);
79     update(update, x1, x2, op, 1);
80     y0 = y;
81 }

```

Picture

```

1  const int maxn = 5010 << 3;
2
3  struct {
4      int l, r, len, cnt, lflag, rflag, num;          //不同点  num为不相交的线段数量, flag用来合并
num
5  } tr[maxn];
6
7  void solve() {
8      int n;
9      cin >> n;
10     vector<int> X;
11     struct node {
12         int y, x1, x2, op;
13         bool operator< (const node& N) const {
14             if (y == N.y) {
15                 return op > N.op;
16             }
17             return y < N.y;
18         }
19     };
20     vector<node> seg;
21     for (int x1, y1, x2, y2; n--;) {
22         cin >> x1 >> y1 >> x2 >> y2;
23         X.push_back(x1), X.push_back(x2);
24         seg.push_back({y1, x1, x2, 1}), seg.push_back({y2, x1, x2, -1});
25     }
26     sort(X.begin(), X.end());
27     X.erase(unique(X.begin(), X.end()), X.end());
28     sort(seg.begin(), seg.end());
29
30     auto pushup = [&](int p) {          //不同点
31         if (tr[p].cnt) {                //有覆盖
32             tr[p].len = X[tr[p].r + 1] - X[tr[p].l];
33             tr[p].rflag = tr[p].lflag = tr[p].num = 1;
34         } else if (tr[p].l == tr[p].r) { //叶子节点
35             tr[p].len = tr[p].num = tr[p].lflag = tr[p].rflag = 0;
36         } else {
37             tr[p].len = tr[p << 1].len + tr[p << 1 | 1].len;
38             tr[p].num = tr[p << 1].num + tr[p << 1 | 1].num;
39             tr[p].lflag = tr[p << 1].lflag, tr[p].rflag = tr[p << 1 | 1].rflag;
40             if (tr[p << 1].rflag && tr[p << 1 | 1].lflag) tr[p].num--; //重复计算
41         }
42     };
43
44     function<void(int, int, int)> build = [&](int l, int r, int p) {
45         tr[p] = {l, r, 0, 0};
46         if (l == r) {
47             return;
48         }
49         int mid = l + r >> 1;

```

```

50     build(l, mid, p << 1), build(mid + 1, r, p << 1 | 1);
51 };
52 function<void(int, int, int, int)> update = [&](int l, int r, int p, int op) {
53     if (l <= X[tr[p].l] && X[tr[p].r + 1] <= r) {
54         tr[p].cnt += op;
55         pushup(p);
56         return;
57     }
58     int mid = tr[p].l + tr[p].r >> 1;
59     if (l < X[mid + 1]) update(l, r, p << 1, op);
60     if (X[mid + 1] < r) update(l, r, p << 1 | 1, op);
61     pushup(p);
62 };
63 build(0, X.size() - 2, 1);
64 int ans = 0, y0 = seg[0].y, len0 = 0, op0 = 1;
65 for (auto [y, x1, x2, op] : seg) { //一定要先计算完"加边"再计算"减边",且计算完"加边"要
再计算一次
66     if (y == y0) { //同一高度
67         if (op0 == op) { //都为加边或都为减边
68             update(x1, x2, 1, op);
69             continue;
70         } else { //加边算完计算一次
71             ans += abs(tr[1].len - len0);
72             len0 = tr[1].len, op0 = -1; //变减边
73             update(x1, x2, 1, op);
74         }
75     } else {
76         op0 = 1; //不同高度从加边开始,顺便计算一次
77         ans += abs(tr[1].len - len0) + tr[1].num * (y - y0) * 2;
78         len0 = tr[1].len, y0 = y;
79         update(x1, x2, 1, op);
80     }
81 }
82 ans += len0; //加上没被计算的
83 cout << ans << "\n";
84 }

```